

Tarea 2: Apache Cassandra

Profesor: José Luis Martí Lara
Ayudante: Eduardo Rodríguez Prado

1. Introducción

Apache Cassandra es una base de datos NoSQL distribuida de tipo *wide-column*, diseñada para trabajar con grandes volúmenes de datos distribuidos entre múltiples nodos.

A diferencia de MongoDB, el diseño de Cassandra se encuentra fuertemente orientado a las consultas que realizará la aplicación. Para ello utiliza **claves de partición**, que determinan dónde se almacenan los datos, y **columnas de clustering**, que permiten organizarlos dentro de una partición.

El objetivo de esta tarea es aplicar *Cassandra Query Language* (CQL), modelado orientado a consultas, desnormalización, índices SAI, replicación, niveles de consistencia y alta disponibilidad, comparando finalmente estas características con MongoDB.

2. Contexto del caso

“Voice Managers”, después de la ayuda que brindo utilizando MongoDB, desea evaluar una solución equivalente utilizando Cassandra, para comparar cual es más conveniente.

Para esto será necesario diseñar las tablas considerando previamente las consultas requeridas, pudiendo duplicar información cuando sea necesario. Usted deberá crear la base de datos, realizar consultas, analizar su ejecución y estudiar el comportamiento del clúster ante la caída de nodos.

3. Set Up

Para desarrollar la tarea se utilizará **Apache Cassandra mediante Docker**. Por lo tanto, **no es necesario instalar Cassandra directamente en el computador**.

Antes de comenzar deberá tener instalado:

- **Docker Desktop**, en caso de utilizar Windows.
- Docker y Docker Compose, en caso de utilizar Linux.

En Aula se proporcionará un archivo `docker-compose.yml` que permitirá levantar un clúster de Cassandra compuesto por **3 nodos**.

1. Levantar el clúster

Abra una consola en la carpeta donde se encuentra el archivo `docker-compose.yml`.

En Windows puede utilizar **PowerShell** o la terminal de Windows. En Linux puede utilizar cualquier terminal.

Ejecute:

```
docker compose up -d
```

Espere algunos minutos mientras Cassandra inicia. Posteriormente verifique que los contenedores se encuentren funcionando mediante:

```
docker ps
```

Para comprobar el estado del clúster ejecute, desde la misma consola:

```
docker exec -it cassandra1 nodetool status
```

Si los nombres de los contenedores son diferentes, deberá reemplazar `cassandra1` por el nombre correspondiente.

Los tres nodos deberán encontrarse en estado:

UN (Up/Normal)

Incluya una captura de este resultado en el informe.

2. Acceder a Cassandra

Para ejecutar consultas CQL deberá ingresar a `cqlsh`, el cliente de línea de comandos incluido dentro del contenedor de Cassandra.

Desde **PowerShell** o la terminal del sistema operativo ejecute:

```
docker exec -it cassandra1 cqlsh
```

A partir de este momento aparecerá una consola similar a:

```
cqlsh>
```

Los comandos CQL de la tarea, tales como `CREATE`, `SELECT`, `INSERT` o `CONSISTENCY`, deberán ejecutarse dentro de esta consola.

3. Crear el keyspace

Dentro de `cqlsh`, determine primero el nombre del datacenter:

```
SELECT data_center FROM system.local;
```

Luego cree un keyspace llamado `actors` utilizando `NetworkTopologyStrategy` y factor de replicación 3.

Por ejemplo, si el datacenter se llama `datacenter1`:

```
CREATE KEYSPACE actors
WITH replication = {
    'class': 'NetworkTopologyStrategy',
    'datacenter1': 3
};

USE actors;
```

Si el nombre del datacenter es diferente, reemplace `datacenter1` por el valor obtenido anteriormente.

Importante: los comandos `docker` y `nodetool` indicados mediante `docker exec` se ejecutan desde PowerShell o la terminal del sistema operativo. Las consultas CQL se ejecutan dentro de `cqlsh`.

4. Desarrollo

Elabore un informe PDF que incluya las consultas CQL utilizadas, resultados, capturas necesarias y una breve explicación de cada sección.

Salvo que se indique lo contrario, no utilice `ALLOW FILTERING`.

Preparación de los datos — 10 puntos

Genere e ingrese información ficticia de, al menos, **20 actores o actrices de voz**. Puede utilizar un script o inteligencia artificial para generar los datos.

Cada actor deberá poseer, como mínimo:

- Identificador único, nombre, nacionalidad, fecha de nacimiento y edad.
- Patrimonio estimado.
- Uno o más idiomas almacenados mediante una colección.
- Uno o más personajes.

Cada personaje deberá incluir nombre, producción, tipo de producción (profesional o independiente), rol, cantidad de apariciones y géneros.

Diseño de las tablas

Diseñe las tablas considerando las consultas que se realizarán. Como mínimo, deberá permitir:

1. Obtener un actor mediante su identificador.
2. Consultar actores de una nacionalidad ordenados por edad.
3. Obtener personajes de un actor considerando rol y apariciones.

Para cada tabla indique su **PRIMARY KEY**, clave de partición, columnas de clustering y qué consulta busca resolver.

Se permite desnormalizar y crear tablas adicionales.

Entregue además un archivo `schema.cql` con la creación del keyspace y las tablas.

Parte 1: Consultas — 10 puntos

Utilice CQL. Salvo que se indique lo contrario, limite los resultados a 10, evite `SELECT *` y no utilice `ALLOW FILTERING`.

1. Búsqueda por clave de partición — 2 puntos

Busque un actor utilizando su identificador y muestre nombre, nacionalidad, edad e idiomas. Explique por qué la consulta puede realizarse eficientemente.

2. Consulta por nacionalidad — 3 puntos

Consulte actores de una nacionalidad existente y ordénelos por edad.

Explique qué atributo funciona como clave de partición y cuál permite establecer el orden.

3. Consulta de personajes — 3 puntos

Seleccione un actor con varios personajes y obtenga únicamente aquellos cuyo rol sea **principal**, ordenados según su cantidad de apariciones.

La consulta deberá resolverse mediante la clave primaria de la tabla.

4. LIMIT y ordenamiento — 2 puntos

Obtenga los primeros 5 registros de una partición utilizando el orden definido por una columna de clustering.

Explique por qué Cassandra permite dicho orden y por qué no permite ordenar arbitrariamente como una base de datos relacional.

Parte 2: Procesamiento de consultas y diseño — 40 puntos

1. TRACING — 8 puntos

Active:

```
TRACING ON;
```

Repita las consultas 1, 2 y 3 de la Parte 1.

Analice la información obtenida, identificando cuando corresponda:

- Nodo coordinador y nodos involucrados.
- Operaciones realizadas.
- Tiempo reportado.
- Relación con la clave de partición y clustering.

Incluya evidencia relevante del tracing.

2. Consulta sin clave apropiada — 8 puntos

Sobre la tabla principal de actores, intente consultar utilizando únicamente **Nacionalidad** y **Edad**, sin índices.

Registre el resultado entregado por Cassandra.

Luego repita la consulta utilizando:

```
ALLOW FILTERING;
```

con TRACING ON.

Explique por qué Cassandra restringe este tipo de consultas y por qué ALLOW FILTERING puede ser problemático con grandes volúmenes de datos. **solo esta parte se puede usar ALLOW FILTERING**

3. Storage-Attached Indexing (SAI) — 10 puntos

Cree índices SAI sobre **Nacionalidad** y **Edad**. La estructura general es:

```
CREATE INDEX nombre_indice
ON nombre_tabla (atributo)
USING 'sai';
```

Realice consultas:

1. Por nacionalidad.
2. Por edad.
3. Combinando ambos atributos.

Luego cree un índice SAI sobre la colección de idiomas y utilice **CONTAINS** para buscar actores según un idioma.

Compare brevemente estas consultas con una búsqueda mediante clave de partición y explique por qué no conviene indexar indiscriminadamente todos los atributos.

4. Desnormalización vs. índices — 7 puntos

Compare una consulta por nacionalidad realizada:

1. Mediante la tabla diseñada específicamente para esa consulta.
2. Mediante la tabla principal y un índice SAI.

Analice el uso de la clave de partición, duplicación de información, costo de mantención e índices adicionales.

5. Escalabilidad — 7 puntos

Suponga que existen **10 mil millones de registros**.

Explique cómo se verían afectadas:

- Consultas mediante clave de partición y clustering.
- Consultas con **ALLOW FILTERING**.
- Consultas mediante SAI.
- Cantidad de particiones o nodos consultados.
- Almacenamiento requerido por desnormalización e índices.

Explique por qué en Cassandra es importante conocer previamente las consultas que realizará la aplicación.

Parte 3: Comparación, alta disponibilidad y actualización — 39 puntos

1. MongoDB vs. Cassandra — 10 puntos

Compare el modelo desarrollado en esta tarea con el utilizado anteriormente en MongoDB.

Considere, como mínimo:

- Documentos embebidos frente a tablas desnormalizadas.
- Flexibilidad de esquema.
- Claves de partición y clustering.
- Índices.
- Consultas no previstas.
- Duplicación de información.
- Escalabilidad horizontal.

Explique por qué el mismo diseño de MongoDB no necesariamente es apropiado para Cassandra y qué relación existe entre las consultas esperadas y el diseño de las tablas.

No se solicita determinar que una tecnología sea siempre mejor que la otra.

2. Alta disponibilidad — 15 puntos

El keyspace posee factor de replicación 3.

Compruebe inicialmente mediante `nodetool status` que los tres nodos se encuentran activos.

Realice una consulta y una inserción utilizando:

```
CONSISTENCY ONE;
```

Luego detenga un nodo y repita las operaciones con:

```
CONSISTENCY ONE;
CONSISTENCY QUORUM;
CONSISTENCY ALL;
```

Posteriormente detenga un segundo nodo y pruebe nuevamente ONE y QUORUM desde el nodo restante.

Registre los resultados en una tabla como la siguiente:

Nodos disponibles	ONE	QUORUM	ALL
3			
2			
1			

Explique los resultados considerando factor de replicación, réplicas disponibles, niveles de consistencia y quórum.

Para $RF = 3$, explique:

$$\left\lfloor \frac{RF}{2} \right\rfloor + 1$$

y la relación observada entre **consistencia y disponibilidad**.

3. Recuperación de nodos — 6 puntos

Reinicie los nodos detenidos y verifique mediante `nodetool status` que vuelvan al estado UN.

Compruebe que los datos insertados durante las pruebas continúan disponibles.

Investigue y explique brevemente:

- *Hinted handoff*.

- Escrituras dirigidas a una réplica temporalmente inaccesible.
- Función del proceso de *repair*.
- Relación entre replicación y tolerancia a fallos.

4. Actualización y datos desnormalizados — 8 puntos

Todos los actores participaron en la nueva serie animada:

“Sansa Ball Race”

Su patrimonio aumentó un **32 %**.

Realice mediante un programa o script una actualización que:

1. Calcule el nuevo patrimonio.
2. Actualice todas las tablas donde este dato se encuentre duplicado.
3. Agregue la participación en “Sansa Ball Race”.

Puede utilizar el mismo personaje para todos.

Para menores de 40 años:

- Rol: principal.
- Apariciones: 3.

Para actores de 40 años o más:

- Rol: secundario.
- Apariciones: 1.

Incluya evidencia antes y después de la actualización.

Explique por qué la desnormalización dificulta las actualizaciones, qué ventaja entrega y si sería apropiado utilizar BATCH para agrupar miles o millones de actualizaciones diferentes.

Parte 4: Cierre — 1 punto

Una vez obtenidas todas las evidencias, elimine el keyspace:

```
DROP KEYSPACE actors;
```

En caso de utilizar GitHub para adjuntar imágenes, scripts o datos, el repositorio deberá ser público.

5. Sobre la entrega

1. La tarea se realizara en parejas, las cuales pueden buscar mediante un canal habilitado en discord, **solo uno debe hacer la entrega en Aula**.
2. Es necesario entregar un informe en formato PDF que contenga las respuestas a cada sección, las consultas realizadas, sus resultados y capturas que evidencien el proceso.

3. El nombre del archivo de la entrega debe ser “**apellido1-apellido2.[pdf;zip;tag.gz;etc.]**”
4. El informe deberá contener nombre y rol de los estudiantes.
5. Puede ser realizado utilizando Word, L^AT_EX u otra herramienta, siempre que la entrega final corresponda a un archivo PDF.
6. Junto al informe deberán entregarse:
 - `schema.cql`.
 - Archivo o script utilizado para generar e insertar los datos.
 - Script utilizado para realizar la actualización masiva.

En caso de utilizar GitHub para almacenar estos archivos, deberá incluirse el enlace al repositorio dentro del informe.

7. La fecha de entrega será hasta las 23:59 del viernes 09 de octubre.
8. Las consultas relacionadas con la tarea podrán realizarse mediante el servidor de discord o el foro de tareas en Aula.
9. No se responderán consultas realizadas durante las 24 horas previas a la fecha límite de entrega.

6. Entregable, defensa y cálculo de nota

La tarea constará de dos partes a evaluar:

- El entregable disponible en Aula.
- Una defensa realizada posteriormente.

Entregable

El entregable deberá contener como mínimo:

- Informe en formato PDF.
- Archivo `schema.cql`.
- Datos o script utilizado para generar los datos.
- Script utilizado para la actualización solicitada.

En caso de utilizar un repositorio GitHub, deberá incluirse el enlace dentro del informe **el repositorio debe ser público.**

Defensa

La defensa consistirá en responder una serie de preguntas relacionadas directamente con los contenidos utilizados durante el desarrollo de la tarea. Se realizarán aproximadamente entre 5 y 10 preguntas cortas.

Dependiendo de la respuesta se asignará:

- 0 puntos: respuesta errónea o no responde.
- 0.5 puntos: respuesta parcialmente correcta.
- 1 punto: respuesta correcta.

Las preguntas de la defensa podrán abordar, entre otros temas:

Cassandra Query Language (CQL); Keyspaces; Factor de replicación; Clave primaria; Clave de partición; Claves de partición compuestas; Columnas de clustering; Ordenamiento dentro de una partición; Colecciones; Desnormalización; Modelado orientado a consultas; `ALLOW FILTERING`; `TRACING`; Storage-Attached Indexing (SAI); Escalabilidad; Replicación; Niveles de consistencia; `ONE`, `QUORUM` y `ALL`; Alta disponibilidad; Hinted handoff; Repair; Diferencias entre Cassandra y MongoDB.

Las defensas son individuales.

Cálculo de nota

La nota se calculará de la siguiente manera:

$$NF = I \cdot N_D$$

donde I corresponde a la nota obtenida a partir del puntaje del informe y N_D se calculará de la siguiente forma:

$$N_D = \frac{C_r}{C_t}$$

donde:

- C_r corresponde a la cantidad de puntos obtenidos durante la defensa.
- C_t corresponde a la cantidad total de preguntas realizadas.

Notar que N_D puede ser 0.

8. Descuentos

A continuación se detallan los posibles descuentos:

- Si no se respeta la entrega en formato PDF o el nombre del archivo, se realizará un descuento de **15 puntos**.
- Si el archivo no contiene nombre y rol, se realizará un descuento de **30 puntos**.

- Se descontarán **10 puntos** por cada hora de atraso o fracción, aproximada hacia arriba, en la entrega del informe en Aula.
- No entregar el archivo `schema.cql` tendrá un descuento de **30 puntos**.
- No entregar los datos o el script utilizado para generarlos tendrá un descuento de **30 puntos**.
- No entregar el script correspondiente a la actualización solicitada tendrá un descuento de **20 puntos**.
- En caso de utilizar GitHub y que el repositorio no sea público, se aplicará la sanción correspondiente si no es posible acceder al material requerido.
- Si las imágenes necesarias para evaluar una sección no son accesibles, dicha sección podrá no ser evaluada.
- Llegar tarde a la defensa tendrá un descuento de 5 puntos. Además, el estudiante contará con menos tiempo disponible para responder.
- Los horarios de las defensas se bloquearan el mismo día de la defensa, en ese momento no podrán re agendarla, y en caso de hacerlo tendrán un descuento de **15 puntos**
- Si por falta de tiempo no se alcanzan a realizar todas las preguntas, las preguntas restantes se considerarán con puntaje 0.
- No agendar un horario de defensa implica nota 0 inmediata.
- No asistir a la defensa implica nota 0 inmediata.